helix_proxy — Feature Status Summary

Revision: 1 **Last modified:** 2026-07-01T12:30:00Z **Authority:** §11.4.56 two-audience companion to `Status.md`. Inherits `constitution/Constitution.md` per §11.4.35.

Page 1 — For the product / operations team (plain language)

What is this? helix_proxy is a self-hosted proxy server. Apps and users point their web traffic at it; it forwards that traffic to the internet, can route it through a VPN, and remembers ("caches") repeated downloads so they come back faster.

What works today, proven with real evidence:

- **Web proxying works.** Plain web (HTTP) and secure (HTTPS) requests both go through the proxy and reach the internet. We proved it with real requests that came back stamped by our proxy (a `Via: proxy-squid` marker only our proxy adds).
- **SOCKS5 proxying works.** The second proxy type (SOCKS5, used by many apps) forwards both plain and secure traffic. Proven with real requests.
- **Caching works.** Ask for the same file twice and the second copy is served from the proxy's own memory — confirmed in the proxy's own access log as a real cache "HIT", not a guess.
- **Status/health command works.** The `./status` tool truthfully reports which parts are running, which ports are open, and whether connections succeed.
- **The certificate checker works.** The piece that will validate our future auto-HTTPS certificates already passes its full self-test (37/37) and is deliberately built so it cannot rubber-stamp a bad certificate.

What's built but not switched on yet (in progress):

- The **smart VPN-aware routing brain** (the "control-plane") — the part that decides which VPN tunnel each request should use, plus the admin dashboard, live health publisher, and metrics. The code exists and passes its component tests, but it is not yet plugged into the live proxy, so end users cannot use

it yet. The admin page you see today is still a temporary placeholder.
- **Username/password protection** on the proxy and **live dashboards/metrics** — designed and partially tested, not switched on.
- **Automatic HTTPS certificates (Let's Encrypt)** — the checker is done; actually issuing and renewing certificates is the next step.

What needs a decision or action from you (operator):

- To prove **VPN routing** end-to-end we need a real VPN tunnel up and the expected exit IP provided. Until then the test honestly skips rather than pretend.
- To turn on **Let's Encrypt for a real domain** we need you to choose a DNS provider, supply a scoped API token, and authorise go-live for the real domain.

Bottom line: the core proxy (forward HTTP, HTTPS, SOCKS5, caching) is working and evidence-backed. The advanced VPN-routing/admin/metrics layer and real HTTPS issuance are still being wired in. Nothing is claimed "done" without a real captured proof behind it.

Page 2 — For software engineers

Scope. Reconciled against branch `feature/vpn-aware-dynamic-routing` @ `bf38c35`. Deployed stack = `docker-compose.yml` (Squid `:53128`, Dante `:51080`, `proxy-admin` = `traefik/whoami` placeholder `:58080`, `proxy-vpn` observed Stopped). Control-plane = separate Go module `digital.vasic.helixproxy/controlplane`, not yet in the byte path.

Tally: 9 PASS-with-evidence · 10 PENDING · 3 OPERATOR-BLOCKED. No FAIL; no metadata-only PASS.

PASS-with-evidence (deployed data plane + LE Phase 1):

Feature	Decisive evidence
HTTP forward proxy	`qa-results/challenges/20260701T085518Z/evidence/http/forward_http_evidence.txt` — proxy=204, `Via: 1.1 proxy-squid (squid/6.13)`
HTTPS CONNECT tunnel	same file, `https_200` sub-probe (`Connection established` → upstream 200)
SOCKS5 + SOCKS5-over-HTTPS	`qa-results/challenges/20260701T085518Z/evidence/socks5/socks5_evidence.txt` (204 + 200)

Feature	Decisive evidence
Response caching	qa-results/comprehensive/cache_hit.evidence (TCP_MEM_HIT/200) + squid_access_snapshot.log
Cache admin (cachectl)	qa-results/comprehensive/cache_cmd_stats.out
ACL allow-path	forward challenge (localnet admit) — deny-path PENDING
./status health	qa-results/comprehensive/status.out + status_json.out
Admin /health reachability	qa-results/comprehensive/after_run.log:92 (placeholder-qualified)
LE cert-analyzer	qa-results/regression/cert_analyzer_selfvalidation/* (37/37 + §1.1 RED→GREEN)

PENDING (code/tests exist, not end-user-reachable): VPN-aware dynamic routing (internal/routing, golden-config unit/integration GREEN, p4-compiler); Control API REST/SSE/PAC/mTLS/metrics (internal/api + cmd/api, unit -race + real-PG integration + §1.1 mTLS mutation, p6-api); VPN health publisher (internal/vpn + cmd/healthd, real-gluetun integration + chaos + §1.1, p3-healthd); circuit breaker/tier-failover (internal/breaker, p5b-breaker); ACL helper graceful-503 (internal/aclhelper, p5-aclhelper); PAC generation (internal/pac golden); proxy auth 407 / ACL deny-path (auth_407 analyzer self-validated at fixture level only); observability/metrics (config/prometheus/prometheus.yml is an explicit NOT-YET-LIVE PLAN, targets absent); LE issuance/renewal Phases 2/3/5. Live stress/chaos/ddos/benchmark/memory/concurrency runs SKIP (suite_results.txt: PASS=0 SKIP=6 FAIL=0) — dynamic stack (P10) not deployed; analyzers self-validated + RED regression guards (ddos_flood_evidence, benchmark_baseline_ratchet) PASS.

OPERATOR-BLOCKED: LE Phase 4 (staging real DNS-01 — needs DNS provider + scoped token as Podman secret); LE Phase 6 (production cutover — operator go-live gate); real VPN egress proof (needs live tunnel + VPN_EXIT_IP; proxy-vpn Stopped; the old host_ip==proxy_ip "VPN verified" bluff was removed — tests/verify-proxy.sh:87-98).

Key anti-bluff notes. (1) HelixQA live exec SKIPs honestly — helixqa binary unbuildable (6 own-org sibling modules absent), proxy re-proven via a stdlib HTTPExecutor replica (qa-results/helixqa/*/mech_*.txt). (2) Cache challenge client-side SKIP is topology_unsupported (rootless subuid owns access.log); the TCP_MEM_HIT PASS comes from comprehensive-test's container-side snapshot. (3) Every analyzer (no_leak, graceful_503, egress_neq_host, xcache_hit, auth_407, cert-analyzer) is golden-good/

golden-bad self-validated (§11.4.107(10)). (4) Video-confirmation is N/A — helix_proxy is headless; proof is captured status codes + `Via:` headers + `access.log` `TCP_*HIT`.

Cross-refs: control-plane `control-plane/README.md`; LE `docs/design/letsencrypt/Status.md`; HelixQA bank `tools/helixqa/banks/proxy.yaml`; challenge bank `challenges/scripts/run_proxy_challenges.sh`.